CAPSTONE PROJECT

B. Tech 2022 Admitted Batch - External Review

TITLE : Smart Lot: Adaptive Parking Management System

BRANCH : CSE(AI&ML)

BATCH NUMBER : 07

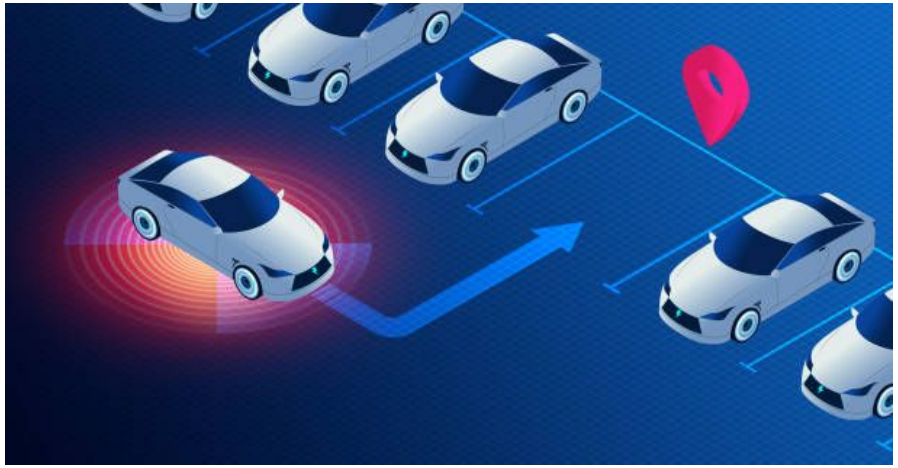
TEAM MEMBERS : R.Yamini (22NU1A4243)
T.Charan (22NU1A4252)
Y.Sai Vardhan(22NU1A4262)
S.Srinivas(22NU1A4244)
M.Karthikeya(22NU1A4235)

GUIDE : Mrs.K. Vandhana

NSRIT

SMART LOT: ADAPTIVE PARKING MANAGEMENT SYSTEM

Introduction: The Strategic Overview



Rapid urban expansion has exposed critical inefficiencies in parking management, resulting in congestion, increased emissions, and driver frustration. Traditional smart parking systems rely on intrusive hardware (e.g., geomagnetic sensors), which are costly, maintenance-heavy, and difficult to scale.

SMART LOT introduces a **non-intrusive, software-defined solution** that transforms existing CCTV/IP camera infrastructure into a real-time parking intelligence system, eliminating the need for dedicated physical sensors while ensuring high scalability and ROI.

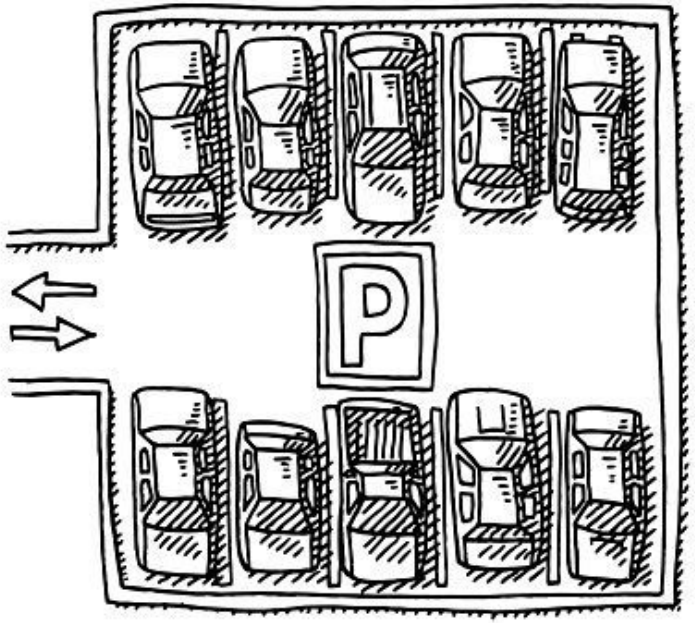
At its core, the system uses a **custom-optimized YOLOv8 model** trained on 12,000+ top-down surveillance images for robust vehicle detection under challenging conditions. A **Digital Twin framework**, implemented with Shapely, maps parking spots as ROI polygons and determines occupancy via bounding box intersection, with a 15% threshold for state changes.

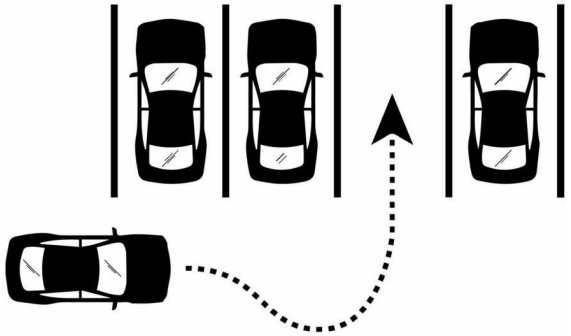
The platform is built on a **FastAPI + SQLAlchemy architecture**, managing hierarchical entities (Lots, Zones, Spots) within a relational database (SQLite/PostgreSQL). A key innovation, the **Matrix Canvas**, enables multi-camera spatial alignment through a unified coordinate system, with **Virtual Portals** ensuring continuity across large facilities.

Introduction: The Strategic Overview (Cont'd)

For user interaction, SMART LOT features a **Dijkstra-based routing engine** that models parking layouts as graphs to guide drivers to the nearest available spot via optimal paths. The system includes a **tactile HUD**, haptic-audio feedback, and a **"Find My Car"** module for reverse navigation.

In essence, SMART LOT combines computer vision, relational database engineering, and advanced graph theory. It sets a new benchmark for cost-optimized urban facility management, proving that a software-first approach can deliver higher accuracy and better user experiences than traditional, hardware-heavy alternatives.



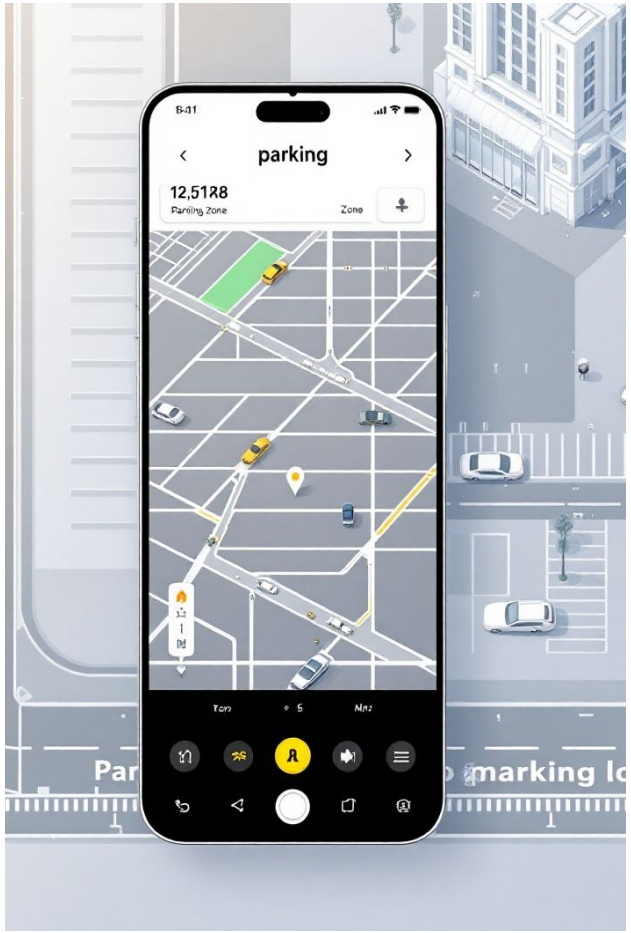


ABSTRACT

The SMART LOT project introduces a cost-optimized, software-defined paradigm for urban mobility, designed to bridge the gap between static surveillance and real-time infrastructure. While traditional parking systems rely on expensive, maintenance-heavy hardware sensors, SMART LOT leverages high-performance computer vision to transform standard CCTV feeds into a dynamic "Digital Twin" of the parking landscape. The system's core is powered by a custom-trained YOLOv8 inference engine, fine-tuned on thousands of varied-perspective frames to ensure robust vehicle detection and spot-level occupancy classification via high-precision geometric ROI (Region of Interest) intersection analysis.

A significant architectural evolution in this version is the transition to a Relational Master-Graph framework. Utilizing a FastAPI backend and SQLAlchemy persistence, the system now supports multi-lot scalability and complex architectural mapping. A unique "Matrix Canvas" allows administrators to visually stitch multiple camera zones into a unified global coordinate system using transformation matrices and "Virtual Portals."

The end-user experience is delivered through a Tactile Driver HUD, which moves beyond simple Euclidean distance to provide immersive, Dijkstra-powered shortest-path navigation. This navigational engine integrates real-time database telemetry to route drivers to the nearest verified empty spot while avoiding reserved zones. Key innovations include haptic-audio feedback for status updates and a dedicated "Find My Car" reverse-routing logic for seamless vehicle retrieval. By converting passive surveillance into active intelligence, SMART LOT demonstrates a 70-80% reduction in deployment costs, offering a scalable, cloud-ready foundation for the integrated smart cities of the future.



Problem Statement:

Lack of reliable, real-time spot-level parking data is a major urban bottleneck, causing congestion, high search times (over 15 minutes), and increased CO2 emissions. Existing sensor and basic camera solutions are either costly or fail under changing conditions. This project provides a cost-effective, robust computer vision solution that not only ensures accurate occupancy data but also transforms it into a Driver Guidance and Navigation Service for real-time routing to the nearest open spot. This fully addresses both the data reliability and internal navigation bottlenecks for maximum efficiency.

Literature Survey

The development of smart parking systems has gained significant attention in recent years due to increasing urban congestion and inefficient parking space utilization. Early approaches primarily relied on network-based communication and infrastructure-heavy solutions. Lu et al. [1] proposed the SPARK system, a VANET-based parking framework designed for large-scale parking environments, enabling real-time communication between vehicles and infrastructure. While effective, such systems require extensive vehicular network integration, limiting their practical deployment.

With the advancement of Internet of Things (IoT), numerous studies have focused on sensor-based parking management systems. Recent IEEE-based implementations [2], [3] demonstrate the use of IoT technologies for real-time parking monitoring and control. Pawar et al. [4] presented a comprehensive study on smart parking technologies, highlighting the use of ultrasonic and magnetic sensors for occupancy detection. Similarly, Kaur and Malhotra [6] reviewed IoT-based parking systems and emphasized their ability to provide real-time monitoring and automation. However, these systems are often associated with high installation and maintenance costs due to their dependence on physical sensors.

Cloud-integrated solutions have also been explored to enhance scalability and centralized management. P. B. et al. [5] introduced a cloud-based IoT parking model that enables efficient data processing and system coordination. While such architectures improve scalability, they continue to rely on hardware-based sensing mechanisms, which increases system complexity and cost.

Recent advancements have introduced hybrid systems combining IoT with intelligent features such as automated license plate recognition and advanced analytics. Pradhan et al. [8] proposed an IoT-integrated parking system with automated license plate recognition and payment management, improving both security and user convenience.

Literature Survey (Cont'd)

Abbas et al. [9] further explored intelligent parking solutions using advanced learning-based approaches to enhance decision-making and system adaptability. Despite these improvements, the dependency on hardware infrastructure remains a significant limitation. In parallel, architectural advancements have been proposed to standardize smart parking systems. Ozkaya and Turunc [7] introduced a reference architecture emphasizing modularity, scalability, and interoperability. Such frameworks provide a strong foundation for system design but do not fully address cost optimization and ease of deployment.

More recently, computer vision-based approaches have emerged as a promising alternative to traditional sensor-based systems. Vision-based parking detection systems utilize surveillance cameras to identify vehicle presence, eliminating the need for dedicated hardware sensors. Object detection models such as YOLO (You Only Look Once) [10] have demonstrated high efficiency and accuracy in real-time detection tasks, making them suitable for parking applications.

The availability of large-scale annotated datasets has further enhanced the performance of vision-based systems. The PKLot dataset [11] provides extensive parking lot images captured under diverse environmental conditions, supporting robust model development. Additionally, recent work on vision-based parking slot detection [12] demonstrates the effectiveness of deep learning techniques in accurately identifying parking occupancy using camera-based inputs.

Building on these advancements, the SMART LOT system adopts a vision-based approach, eliminating sensor-based infrastructure. It uses optimized object detection and geometric intersection analysis for accurate parking occupancy detection. Spatial mapping and a digital twin enable efficient multi-zone coordination and real-time monitoring, addressing scalability, cost, and deployment challenges. Thus, it bridges traditional IoT parking solutions and modern intelligent infrastructure with a scalable, cost-effective, vision-driven framework.

Proposed Work:

The proposed Adaptive Parking Management System utilizes advanced computer vision and target detection architectures to provide accurate, real-time parking space occupancy detection and intelligent driver guidance. Unlike traditional sensor-based systems, the proposed system primarily relies on neural networks and geometric spatial analysis to classify parking spots as occupied or vacant with high precision.

ADVANTAGES:

- Precision Occupancy Detection: Utilizes ROI-based geometric intersections to ensure high accuracy even in complex environments.
- Infrastructure Transformation: Leverages existing CCTV/IP cameras, eliminating the need for expensive physical ground sensors.
- Dynamic Scalability: A modular, software-driven architecture that can be deployed across any number of lots with minimal configuration.
- Optimized Urban Flow: Reduces traffic congestion, fuel consumption, and driver frustration through automated spot discovery.

SOFTWARE REQUIREMENTS

For developing the application, the following are the primary Software Requirements:

- Python 3.10+: The core programming language for backend logic and AI processing.
- OpenCV: Library for real-time computer vision and video frame manipulation.
- YOLOv8 (Neural-Network Framework): Used for high-speed machine learning-based vehicle detection.
- SQLAlchemy: ORM for managing database schema and transactions.
- SQLite / PostgreSQL: Databases for local storage and scalable cloud deployment.
- Shapely & NumPy: Used for spatial analysis and geometric computations.
- FastAPI: A modern, high-performance web framework for building the system's REST API.
- Uvicorn: An ASGI web server implementation for Python.
- PyJWT & Passlib (Bcrypt): Enable secure authentication and password hashing.
- Python-Multipart: Supports file and form-data handling in API requests.

OPERATING SYSTEM SUPPORTED

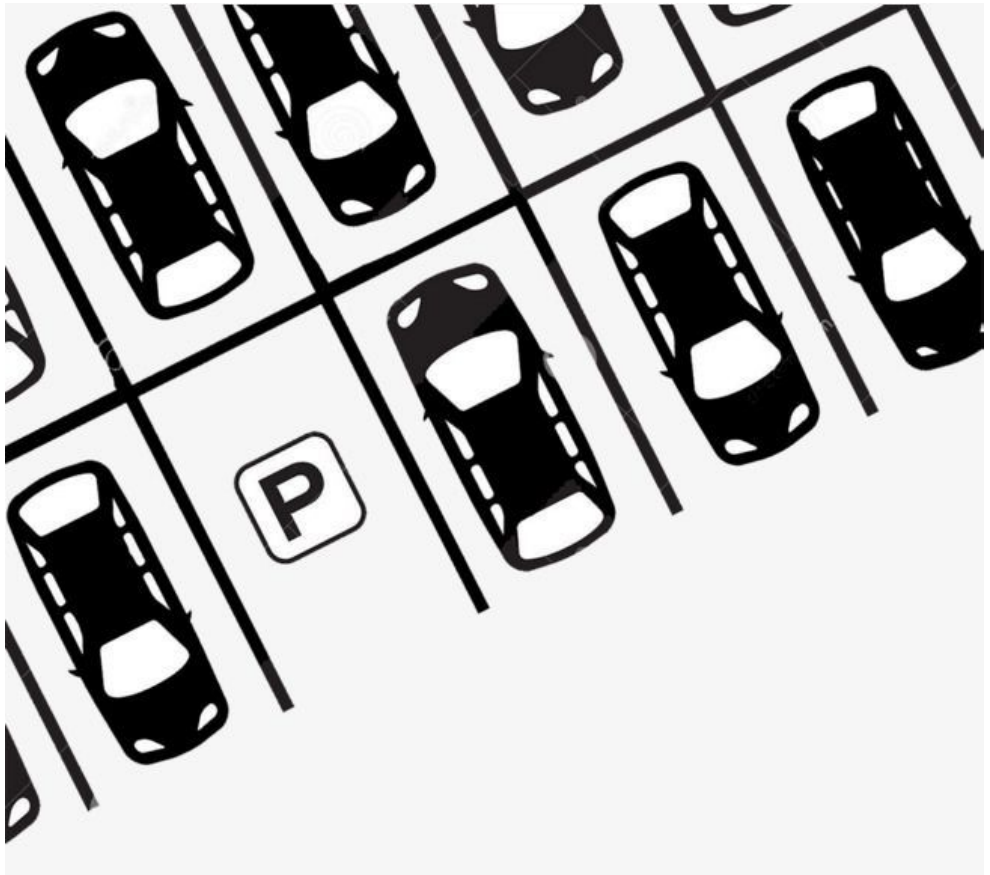
- Windows (Developer and Admin environment)
- Linux (Optimized for server deployment/edge environment)

TECHNOLOGIES AND LANGUAGES USED

- Python: Core backend and visual inference logic.
- HTML5: Frontend structure and semantics.
- CSS3 / Vanilla CSS: Shared design system with high-fidelity cyber-aesthetics.
- JavaScript (ES6+): Frontend interactivity and real-time state synchronization.
- SQL (SQLite / PostgreSQL): Relational query language used to manage the complex, persistent site hierarchy (Lots ➔ Zones ➔ Spots).
- JWT (JSON Web Tokens): Secure, industry-standard authentication protocol used to protect the Administrator Management Hub and API endpoints.

HARDWARE REQUIREMENTS

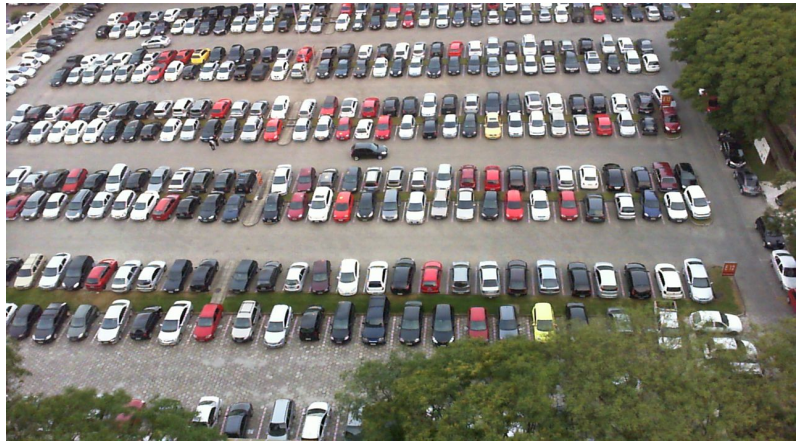
- Processor: Intel i5 / i7 or higher (or equivalent multi-core processor for parallel vision pipelines)
- RAM: 8GB minimum (16GB recommended for handling concurrent high-resolution AI streams)
- Graphics: NVIDIA GPU (Optional, but recommended for hardware-accelerated inference)
- Hard Disk Space: 10GB minimum
- Camera Module: CCTV / IP Camera / Digital Video Source for real-time monitoring
- Connectivity: Local Area Network (LAN) or Internet for API communication and client synchronization



Dataset Used

To ensure the model is robust against different conditions, we utilized a massive, industry-standard dataset.

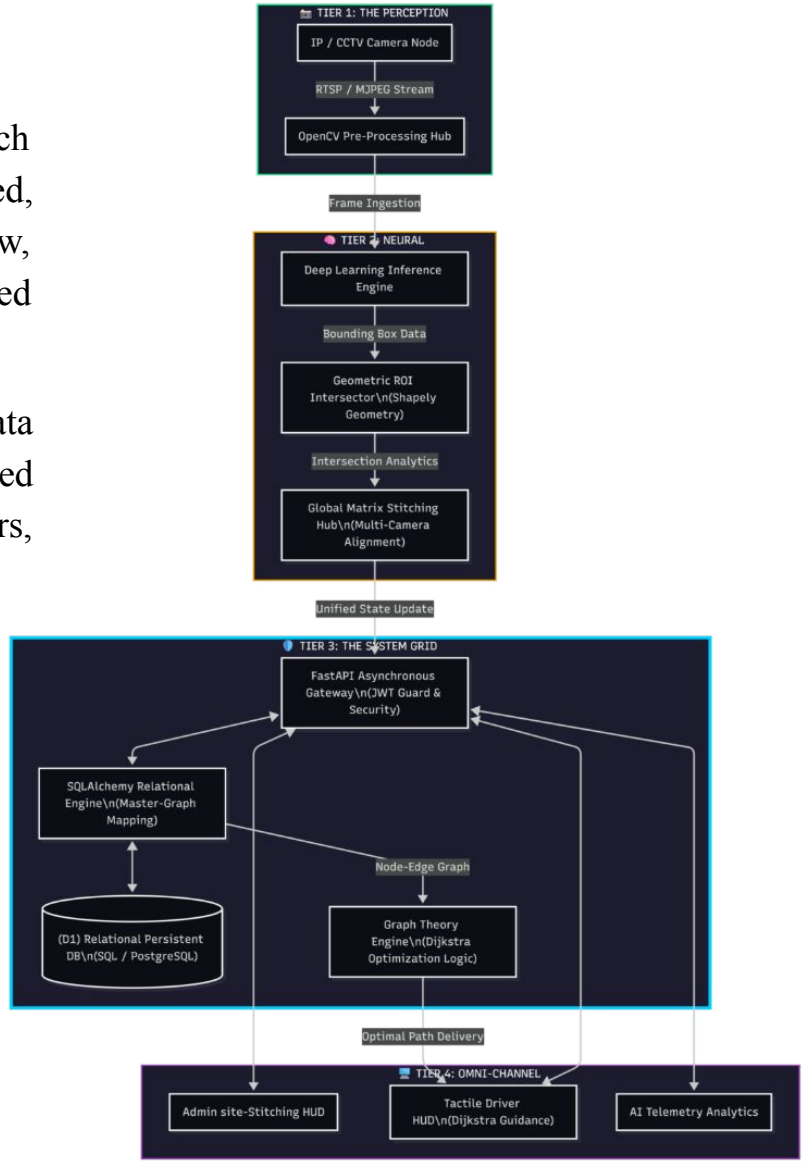
- **Primary Source**: The **PKLot Dataset**, which contains over 600,000 images of parking spots.
- **Environmental Diversity**: The data includes images captured in **Sunny** (heavy shadows), **Cloudy** (neutral lighting), and **Rainy** (reflections) weather.
- **Extracted Patches**: For our specific environment, we verified a local foundation of **911,680 total patches** (484,948 Empty and 426,732 Occupied).
- **Training Subset**: The current model was trained on a balanced subset of **10,000 images** to optimize processing speed without sacrificing accuracy.



SYSTEM ARCHITECTURE

The SMART LOT system architecture is structured as a 4-layer distributed pipeline, where each tier is responsible for a distinct stage in the transformation of raw visual input into structured, real-time parking intelligence. The architecture follows a strictly sequential and modular flow, beginning with video acquisition and progressing through AI-based interpretation, centralized system management, and finally user interaction.

Each tier operates independently while remaining interconnected through well-defined data transitions such as frame ingestion, bounding box propagation, state updates, and graph-based query responses. The system diagram illustrates how data flows vertically across these layers, forming a continuous processing chain.



Tier 1: The Perception Edge (Raw Data Ingestion Layer)

The first tier of the architecture is the Perception Layer, which is responsible for acquiring and preparing visual data from the physical environment. This layer consists of IP/CCTV camera nodes that continuously stream video data using protocols such as RTSP or MJPEG. The incoming video streams are directed to an OpenCV Pre-Processing Hub, where the raw data undergoes initial transformation. This includes:

- Frame extraction from continuous video streams
- Resolution normalization
- Color space adjustments
- Basic filtering and preparation for inference

SYSTEM ARCHITECTURE (Cont'd)

The output of this layer is a structured sequence of frames that are standardized for downstream processing. These frames are then forwarded through a defined transition labeled Frame Ingestion, which connects the Perception Layer to the next stage of the system.

Tier 2: Neural Intelligence (Spatial AI Processing Layer)

The second tier is the Neural Intelligence Layer, which performs the core analytical operations of the system. This layer converts visual input into spatial and semantic information through a series of processing modules.

The first component in this tier is the Deep Learning Inference Engine, which applies a trained object detection model to identify vehicles within each frame. The output of this process is structured as bounding box data, containing positional and classification information for detected objects.

This bounding box data is then passed to the Geometric ROI Intersector, which performs spatial evaluation using computational geometry. Each parking slot is represented as a predefined polygon, and the system calculates the intersection between detected bounding boxes and these polygons. The result of this computation is a set of intersection analytics, which determine the occupancy status of each parking space.

Following this, the processed spatial data is handled by the Global Matrix Stitching Hub, which is responsible for aligning multiple camera perspectives into a unified coordinate system. Since each camera operates within its own local frame of reference, this module applies spatial transformations to merge these views into a consistent global layout.

The output of this tier is a Unified State Update, representing the current occupancy and spatial configuration of the parking environment. This state is then transmitted to the System Grid layer.

Tier 3: The System Grid (Logic, Security, and Persistence Layer)

The third tier functions as the central control and coordination layer of the architecture. It is responsible for managing system logic, maintaining persistent state, and handling communication between components.

At the entry point of this tier is the FastAPI Asynchronous Gateway, which acts as the primary interface for receiving updates from the Neural Intelligence Layer and serving requests from user-facing applications. This gateway processes incoming data streams and ensures that all interactions occur in a non-blocking, concurrent manner.

SYSTEM ARCHITECTURE (Cont'd)

Connected to this gateway is the SQLAlchemy Relational Engine, which maps system entities into structured database models. These entities include parking lots, zones, parking slots, and spatial relationships. The relational engine communicates with the persistent database, which can be implemented using either SQLite or PostgreSQL, depending on deployment requirements.

The database serves as the central repository for all system data, including:

- Configuration data (lots, zones, ROIs)
- Real-time occupancy states
- Spatial mappings from the stitching process
- Graph structures representing navigable paths

Within this tier, the system also constructs a node-edge graph representation of the parking environment. This graph is processed by the Graph Theory Engine, which applies Dijkstra-based optimization logic to compute navigation paths. The graph is dynamically updated based on real-time occupancy data stored in the database.

The result of this computation is an Optimal Path Delivery, which is forwarded to the user interaction layer for presentation.

Tier 4: Omni-Channel UX (User Interaction Layer)

The fourth tier represents the Omni-Channel User Interface Layer, which provides access to system functionality through multiple specialized interfaces. This layer interacts with the System Grid through API calls to retrieve processed data and deliver it to end users.

This tier consists of three primary components:

1. Admin Site-Stitching HUD

This interface allows administrators to configure and manage the system. It is used to define parking zones, create ROI mappings, and perform multi-camera alignment through the stitching process.

SYSTEM ARCHITECTURE (Cont'd)

2. Tactile Driver HUD

The Driver HUD provides navigation and parking assistance to users. It receives the optimal path output from the System Grid and presents it in an interpretable format, enabling users to navigate toward available parking spaces.

3. AI Telemetry Analytics

This module provides a visualization of system activity, including detection outputs, occupancy states, and system performance data. It serves as a monitoring interface for analyzing system behavior in real time.

System-Wide Data Flow

The complete system operates as a continuous pipeline with clearly defined transitions between tiers:

1. Video streams are captured and pre-processed in the Perception Layer
2. Frames are analyzed in the Neural Intelligence Layer for detection and spatial computation
3. Processed state data is transmitted to the System Grid for storage and graph-based processing
4. Navigation and occupancy results are delivered to the Omni-Channel UX Layer

Each transition in this pipeline is explicitly represented in the architecture diagram through labeled flows such as Frame Ingestion, Bounding Box Data, Intersection Analytics, Unified State Update, and Optiaml Path Delivery.

USE CASE DIAGRAM

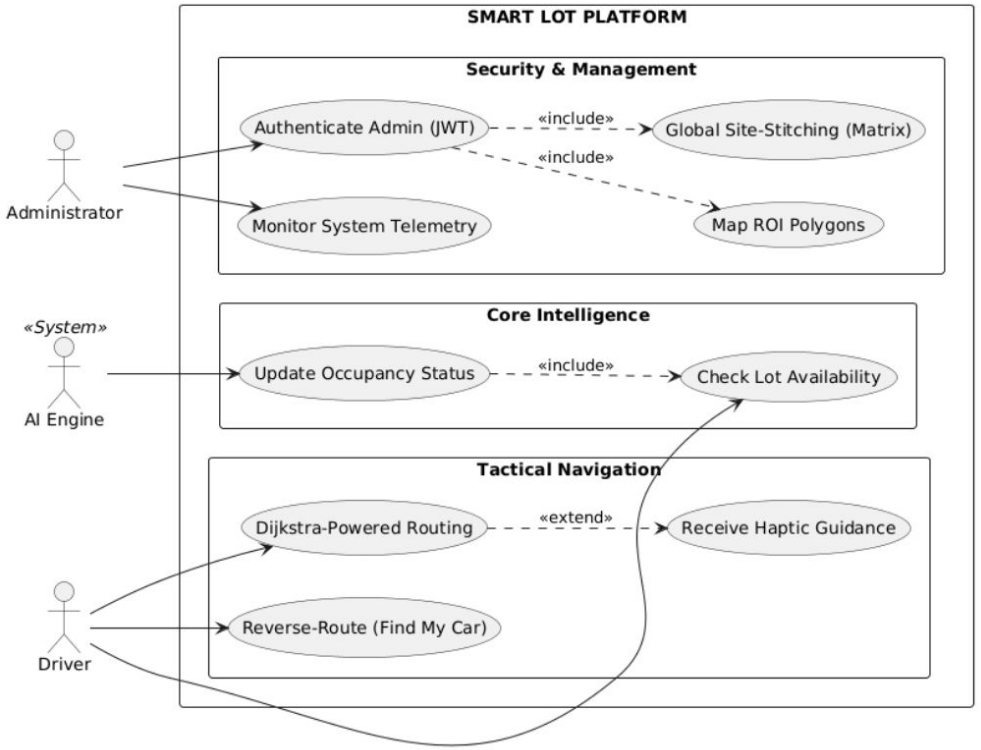
A Use Case Diagram represents interactions between users and the SmartPark system, defining system boundaries and high-level functionalities. It connects the physical parking environment with the digital system through AI-driven vision and spatial analysis.

Actors

- **Driver**: Accesses real-time data for parking and vehicle retrieval.
- **Administrator**: Manages setup, configuration, and system oversight.
- **AI Inference Engine (System)**: Processes video input to detect occupancy and update the database.

Key Use Cases

- **Global Site-Stitching**: Aligns multiple camera zones into a unified coordinate system.
- **Dijkstra-Based Navigation**: Provides the shortest path to an available parking slot.
- **Admin Authentication (JWT)**: Secures system access and configuration.
- **Find My Car (Reverse Routing)**: Guides users back to their parked vehicle.
- **Haptic/Audio Guidance**: Delivers non-visual navigation cues.
- **Configure Parking Slots (ROI)**: Defines parking space boundaries for detection.

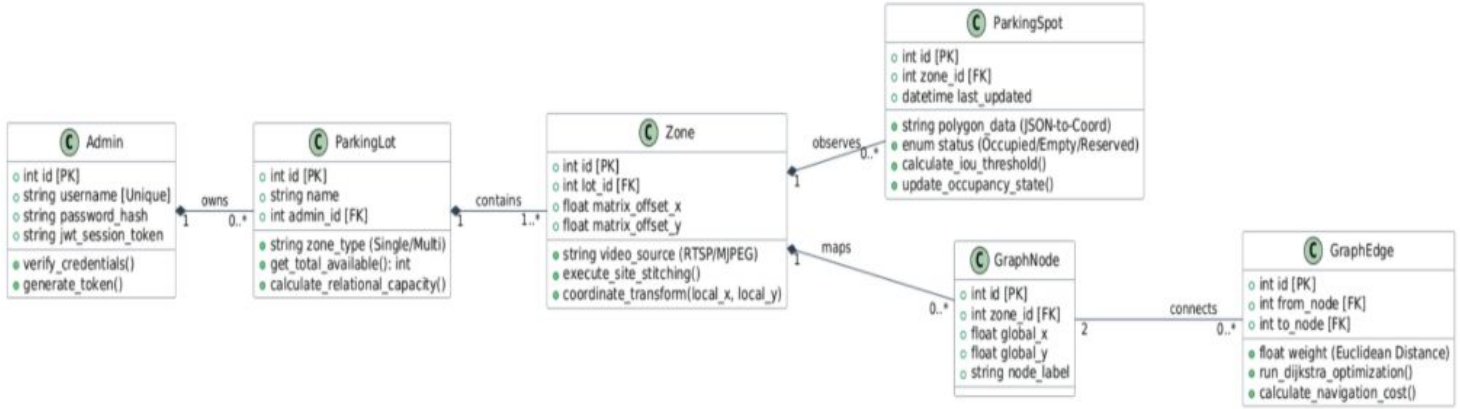


CLASS DIAGRAM

A Class Diagram is a type of UML diagram that represents the static structure of the SMART LOT system. It illustrates the classes, their internal attributes, operational methods, and the relational hierarchy between them. In our project, this diagram serves as the structural blueprint for the Relational Master-Graph, modeling how data such as site-stitching matrices and occupancy states interact across the SQLAlchemy persistence layer. In the context of the SMART LOT system, the class diagram captures key entities ranging from the high-level Admin and ParkingLot to the granular GraphNodes and defines the rigid dependencies required for real-time facility management.

Purpose of the Class Diagram

- Defining Object-Oriented Hierarchy: It provides the foundation for the Python classes implemented in the models.py and database.py modules using a one-to-many relationship (Admin ➔ Lot ➔ Zone ➔ Spot).
- Relational Data Modelling: It defines the attributes required for the persistent SQL store, such as polygon coordinate strings for resolution independence and float-based offsets for site stitching.
- Enabling Graph Theory Logic: By establishing classes for GraphNodes and GraphEdges, the diagram blueprints the mathematical infrastructure needed for the Dijkstra-powered shortest-path navigation.
- ORM Mapping: Each class in the diagram translates directly to our PostgreSQL/SQLite tables. Attributes are mapped to typed SQL columns, ensuring data integrity and type safety across the entire backend.



ACTIVITY DIAGRAM

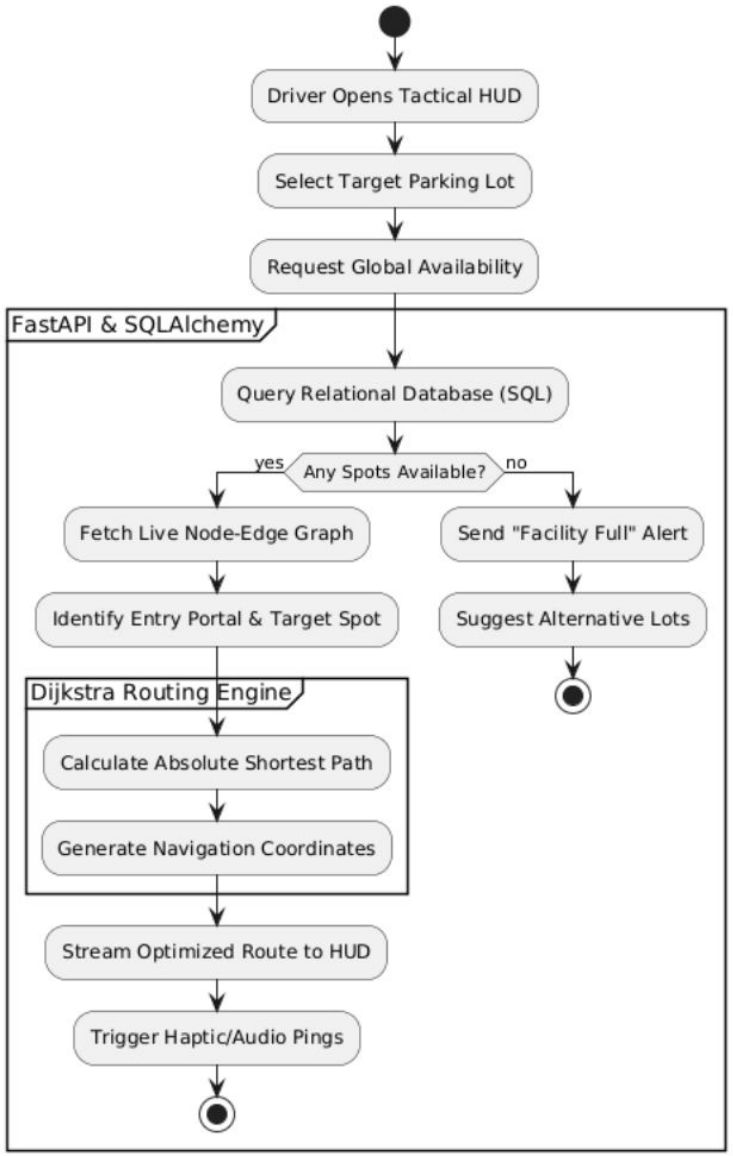
The Activity Diagram represents the SMART LOT system's workflow, outlining the step-by-step process that converts a driver's request into an optimized parking route via the FastAPI backend. It models the Intelligent Driver Guidance use case and serves as the blueprint for the Dijkstra-based routing engine, integrating the driver HUD, SQL database, and graph-based navigation logic.

Key Benefits

- **Optimization Logic:** Shows transition from proximity-based selection to shortest-path routing using Dijkstra's Algorithm
- **Workflow Integration:** Defines interaction between mobile UI, SQLAlchemy layer, and feedback services
- **Architecture Modeling:** Captures decision logic for vacancy checks and portal-to-spot routing

Core Components

- **Action:** Functional steps like path calculation or database queries
- **Decision Node:** Conditional checks such as vacancy availability
- **Partitions (Swimlanes):** Separation of user and backend processes
- **Control Flows:** Directed sequence from request initiation to navigation output



SYSTEM IMPLEMENTATION

The SMART LOT system is implemented as a multi-stage processing pipeline that integrates computer vision, spatial computation, backend services, and user interaction into a unified operational framework. The system processes real-time video streams and converts them into structured parking intelligence, which is then used for monitoring and navigation.

At a high level, the implementation follows a continuous execution cycle consisting of four major stages:

1. Visual Data Acquisition
2. AI-Based Vehicle Detection and Spatial Analysis
3. State Management and Data Persistence
4. User Interaction and Navigation Delivery

5.2.1 Visual Data Acquisition

The implementation begins with the acquisition of live video streams from distributed CCTV/IP camera nodes. These streams are processed using an OpenCV-based pipeline, where frames are extracted at regular intervals.

Each frame undergoes pre-processing operations such as resizing and normalization to ensure compatibility with the detection model. This step ensures that all input data maintains a consistent
format regardless of the source camera.

5.2.2 AI-Based Detection and Spatial Processing

Once frames are prepared, they are passed to the YOLOv8 inference engine, which detects vehicles in real time. The model outputs bounding boxes representing detected vehicles along with confidence scores.

These bounding boxes are then mapped against predefined Region of Interest (ROI) polygons representing parking slots.

SYSTEM IMPLEMENTATION (Cont'd)

The system performs spatial intersection analysis using geometric computation. For each detected vehicle:

- The overlap between the bounding box and ROI polygon is calculated
- If the overlap exceeds a predefined threshold (e.g., 15%), the slot is marked as occupied
- Otherwise, the slot remains vacant

This process is executed for every frame, enabling continuous monitoring of parking occupancy.

5.2.3 State Management and Database Synchronization

After occupancy determination, the system updates the parking state in a centralized relational database. This process involves:

- Identifying the corresponding parking slot
- Updating its occupancy status
- Synchronizing the state across all connected system components

The backend is implemented using a FastAPI asynchronous service, which ensures that updates are processed without blocking ongoing operations.

The system maintains a structured database containing:

- Parking lot configurations
- Zone definitions
- ROI coordinates
- Real-time occupancy states

This database acts as the central reference point for all system operations.

SYSTEM IMPLEMENTATION (Cont'd)

5.2.4 Navigation and User Interaction

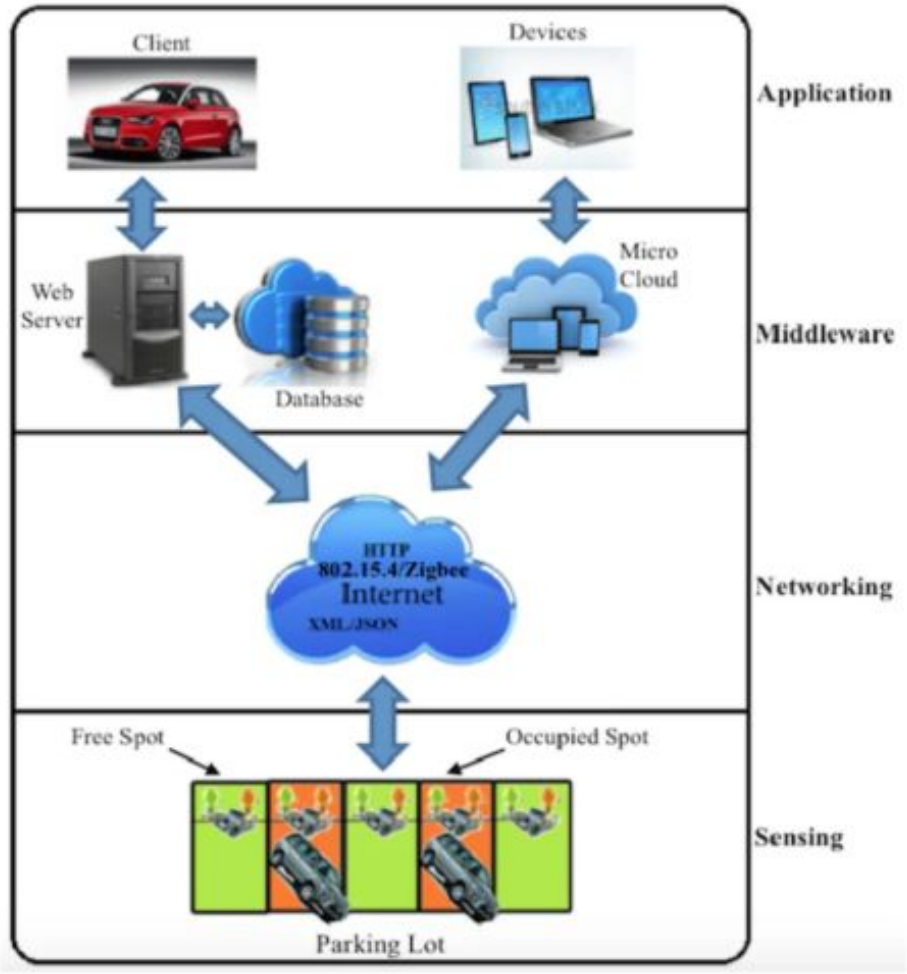
The processed data is made available to user interfaces through API endpoints. When a user requests parking assistance:

- The system retrieves all available parking slots
- Computes optimal navigation paths using graph-based logic
- Sends the results to the user interface

The Driver HUD displays navigation instructions, while the Admin Interface provides system monitoring and configuration tools.

5.2.5 Application Usage Flow

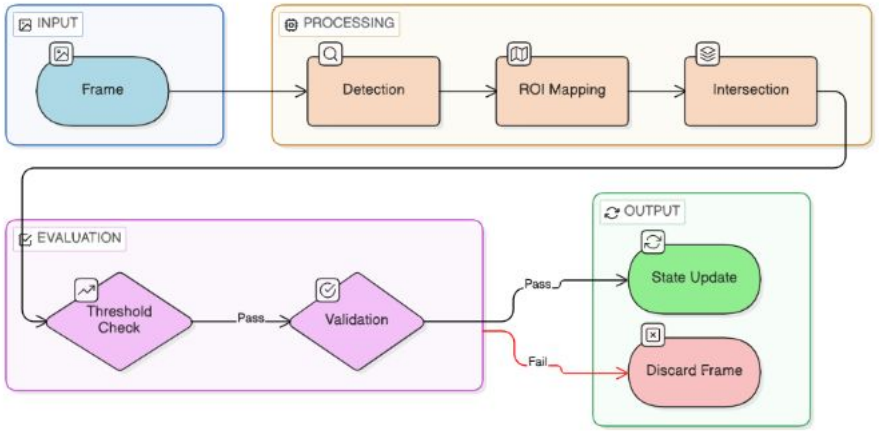
The overall implementation can be summarized as following process:



DETECTION PIPELINE OVERVIEW

The occupancy detection process follows a structured multi-stage pipeline. Each stage contributes to transforming raw visual data into a binary occupancy decision.

1. Frame Acquisition
2. Vehicle Detection (YOLOv8 Inference)
3. Bounding Box Extraction
4. ROI Polygon Mapping
5. Intersection Computation
6. Threshold-Based Classification
7. State Update



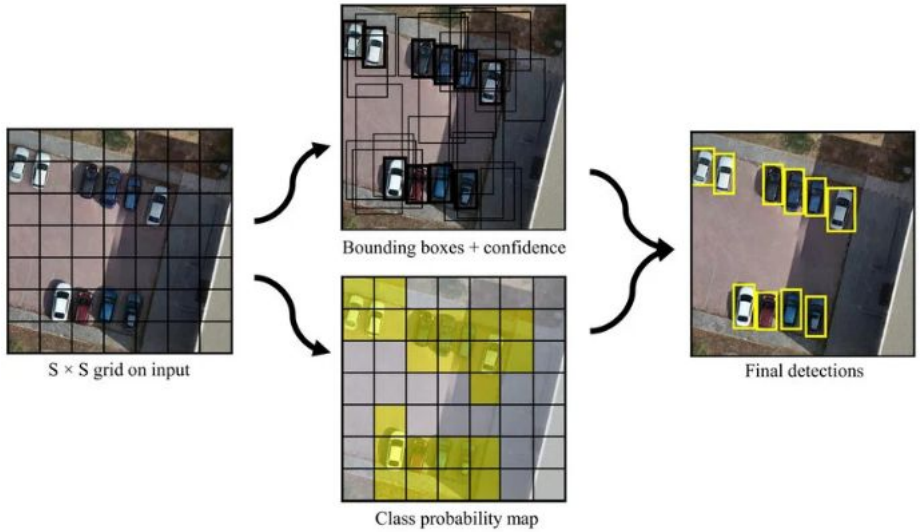
Frame Processing and Detection

The system continuously receives frames from the perception layer. Each frame is passed to the YOLOv8 detection model, which performs object detection in a single forward pass.

The model identifies vehicles and returns:

- Bounding box coordinates `(x1, y1, x2, y2)`
- Confidence score
- Object class

Only detections above a predefined confidence threshold are considered valid.



DETECTION PIPELINE (Cont'd)

ROI-Based Spatial Mapping

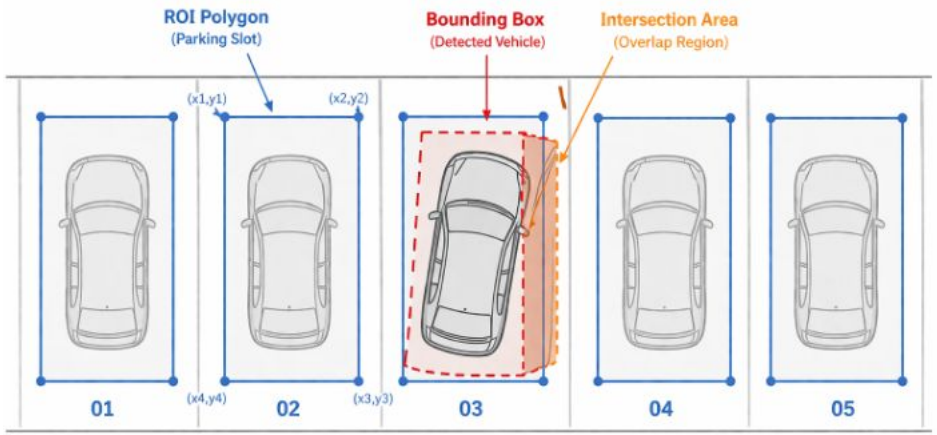
Each parking slot is represented as a Region of Interest (ROI) defined by polygon coordinates. These coordinates are retrieved from the relational database. The system maps detected bounding boxes to these ROIs using geometric comparison.

Intersection Computation

To determine occupancy, the system calculates the intersection area between:

- Vehicle bounding box
- Parking slot polygon

This is performed using geometric operations.

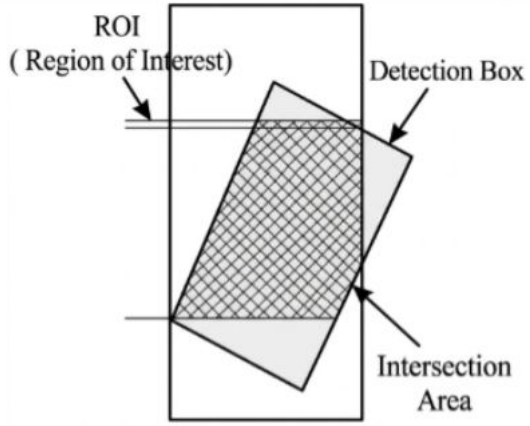


Threshold-Based Occupancy Decision

The system applies a threshold to determine occupancy:

- If occupancy ratio > 0.15 → OCCUPIED
- Else → VACANT

This threshold ensures that minor overlaps, such as passing vehicles, do not incorrectly trigger occupancy.



$$\text{Occupancy Ratio} = \frac{\text{Intersection Area}}{\text{ROI Area}}$$

Handling Edge Cases

The implementation accounts for several real-world scenarios:

1. Partial Overlap
   - Vehicles partially entering a slot may produce small overlap
   - Threshold prevents false positives

DETECTION PIPELINE (Cont'd)

2. Passing Vehicles
   - Vehicles moving across lanes may intersect ROIs briefly
   - Temporal smoothing (next section) filters these cases
3. Occlusion
   - Large vehicles blocking others may affect detection
   - System relies on consistent multi-frame detection
4. Adjacent Slot Interference
   - Bounding boxes overlapping multiple ROIs
   - Highest overlap ratio is prioritized

Multi-Stage Validation Mechanism

To improve detection reliability, an additional validation stage is applied when detection confidence is low.

Components:

1. Color Variance Analysis
   a. Measures pixel intensity variation
   b. Detects presence of texture (vehicle surface)
2. Canny Edge Detection
   a. Identifies structural edges within the ROI
   b. Confirms object boundaries

A slot is marked as occupied only if both conditions are satisfied.

State Synchronization

Once occupancy is determined, the system updates the central database.

Each update includes:

- Slot ID
- Occupancy state
- Timestamp

The update ensures that all system components access consistent data.

The implementation is optimized for real-time operation:

- Detection runs on each frame independently
- ROI computations are lightweight geometric operations
- Parallel processing is used for multiple camera streams

This ensures that the system can handle multiple parking zones simultaneously.

Algorithms Used

1. YOLOv8 Inference: Performs real-time object detection using a single-pass convolutional architecture to localize vehicles within each video frame.
2. Matrix Transformation (Site Stitching): Calculates geometric offsets and perspective shifts to align multiple camera zones into a unified global coordinate system.
3. Intersection over Area (IoA): Measures the precise geometric overlap between detected vehicle bounding boxes and pre-defined Relational ROI Polygons.
4. Temporal Smoothing: Uses a recursive history buffer to eliminate detection "flicker" and ensure stable state transitions in the SQL database.
5. Dijkstra's Shortest-Path Algorithm: Replaces simple linear distance by calculating the mathematically optimal route through the parking facility's Node-Edge Graph.
6. SQL Relational Mapping: Uses SQLAlchemy ORM to manage complex one-to-many relationships between Admins, Lots, Zones, and Parking Spots.
7. JWT Token Validation (Security): Implements Industry-Standard JSON Web Tokens to secure administrative actions and API endpoints.
8. MJPEG Boundary Fragmentation: Enables high-speed, continuous AI telemetry streaming directly to the web dashboard with minimal overhead.
9. Asynchronous API Gateway: Utilizing FastAPI, the system manages high-concurrency requests between the AI engine and the User Viewports.
10. Normalized Coordinate Projection: Translates user-defined points into resolution-independent ratios, ensuring the Digital Twin is accurate across all device types.

INTELLIGENT RECOMMENDATION AND ROUTING IMPLEMENTATION

The intelligent recommendation and routing module is responsible for guiding users to the most optimal parking space within the facility. Unlike basic systems that rely on proximity alone, this implementation models the parking environment as a graph structure, enabling efficient and accurate path computation.

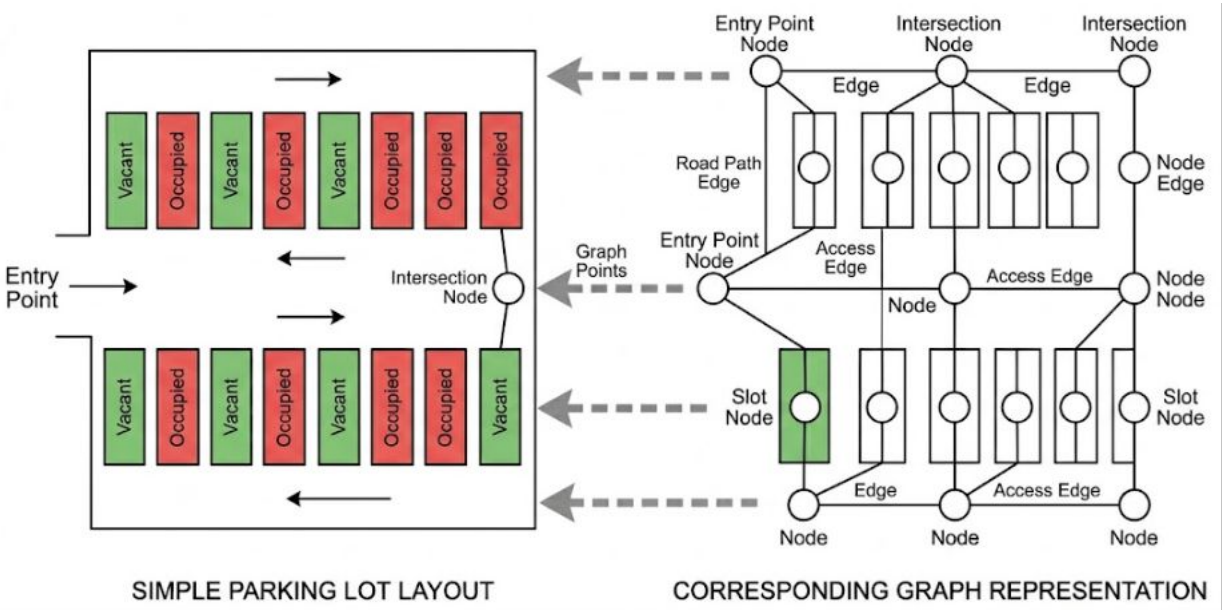
The system continuously evaluates available parking slots and computes the shortest navigable path from an entry point to a selected vacant spot.

5.6.1 Graph-Based Representation of Parking Space

To enable routing, the parking lot is modeled as a graph consisting of nodes and edges:

- Nodes represent key points such as:
  - Entry/exit points (portals)
  - Junctions and intersections
  - Parking slot centroids
- Edges represent:
  - Driveable paths between nodes
  - Distance or traversal cost between locations

This representation allows the system to treat navigation as a mathematical pathfinding problem.



INTELLIGENT RECOMMENDATION AND ROUTING IMPLEMENTATION (Cont'd)

Shortest Path Computation (Dijkstra Algorithm)

To determine the optimal path, the system applies Dijkstra's shortest path algorithm.

Working Principle:

- The algorithm starts from the entry node
- It explores all reachable nodes
- Computes the minimum cost path to each available parking slot
- Selects the path with the lowest total distance

Selection of Optimal Parking Slot, Real-Time Adaptation, and Integration with Backend System

After computing paths to all vacant slots, the system selects:

- The slot with minimum path distance
- The corresponding optimal route

This ensures that the selected slot is not just closest in straight-line distance, but also reachable through the shortest navigable path. And the system continuously updates:

- Parking slot availability
- Graph state

If a selected slot becomes occupied before the user reaches it, the system recomputes the route dynamically using updated data.

The routing module interacts with the backend through:

- Database queries for slot availability
- Graph retrieval and updates
- API endpoints for delivering routing results

The computed path is returned as a sequence of nodes, which is then translated into user-friendly navigation instructions.

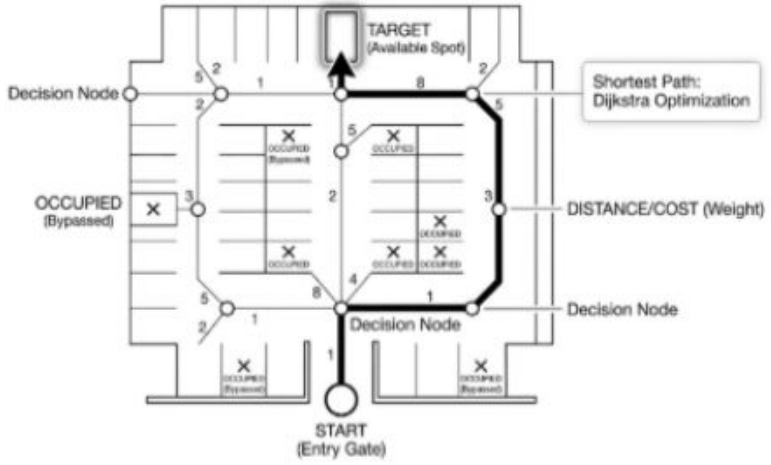


Fig 6.1: Shortest path finding using Dijkstra

DATABASE AND STATE MANAGEMENT IMPLEMENTATION

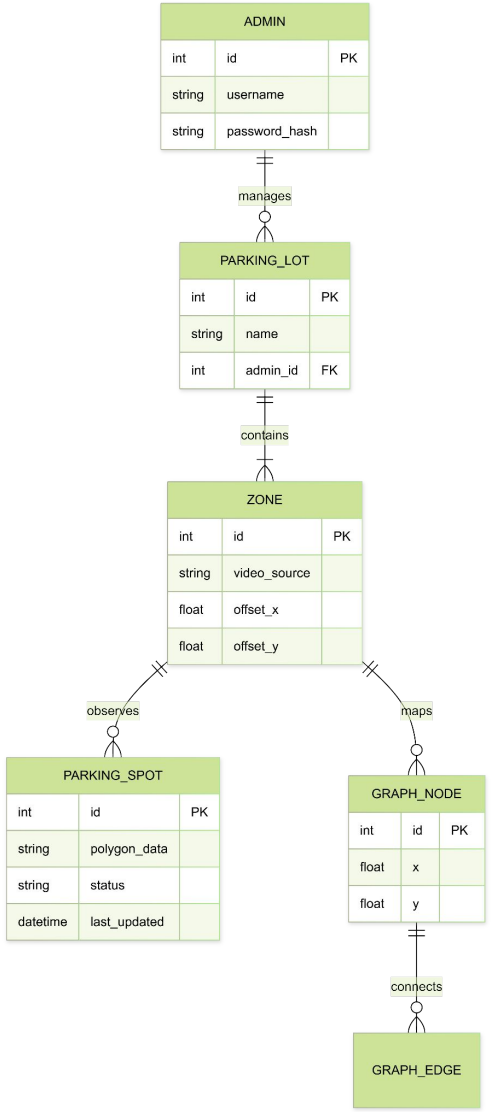
The database and state management component of the SMART LOT system is responsible for maintaining a consistent and real-time representation of the parking environment. It acts as the central coordination layer where outputs from the detection module, routing engine, and user interfaces are synchronized.

The system adopts a relational data model to organize parking infrastructure into structured entities. These include parking lots, zones, individual parking slots, and graph-based navigation elements. Each parking slot is associated with a unique identifier, zone reference, and polygon-based Region of Interest (ROI) coordinates, along with its current occupancy state. Similarly, the routing structure is represented using graph entities, where nodes correspond to navigable positions and edges define connections between them.

To interact with the database, the system uses an Object Relational Mapping (ORM) approach. This allows database tables to be represented as objects within the application, enabling operations such as retrieval and updates to be performed through structured methods rather than direct SQL queries. This abstraction simplifies implementation and ensures consistency across data operations.

Pseudocode: ORM Update Operation

```
session = create_session()
spot = session.query(Spot).filter(id=spot_id)
spot.status = OCCUPIED
session.commit()
```



INTELLIGENT RECOMMENDATION AND ROUTING IMPLEMENTATION (Cont'd)

Whenever a change in parking occupancy is detected, the system performs a controlled state update. Instead of updating the database continuously for every frame, the system first verifies whether the new state differs from the previously stored state. Only confirmed changes are committed to the database, reducing redundant operations and improving efficiency.

Pseudocode: State Update Logic

```
If new_state != previous_state:
    update database
    commit changes
```

The database is integrated with an asynchronous backend service, which manages communication between system components. Detection outputs are sent to the backend, which updates the database accordingly. At the same time, user interfaces and routing modules query the backend to retrieve the latest parking states and spatial data.

This interaction forms a continuous data exchange cycle, where updated occupancy information is immediately available to all connected components. The database ensures that every module operates on the same set of data, preventing inconsistencies between detection results, navigation logic, and user interfaces.

TESTING METHODS

Black Box Testing:

Black Box Testing was performed at the user interaction level, focusing on system behavior without considering internal implementation. The testing covered major functional flows such as Admin Authentication using JWT, Site Stitching operations, Dijkstra-based navigation, and vehicle retrieval mechanisms. Various scenarios were tested, including invalid login attempts, unauthorized API access, and system edge cases such as full parking capacity. The system correctly handled all cases by restricting unauthorized actions and providing accurate real-time responses through the Driver HUD.

White Box Testing:

White Box Testing focused on validating internal system logic across the **Intelligence Core and System Grid layers**. The geometric occupancy detection using IoA was verified to ensure correct threshold-based classification of parking slots. The Dijkstra routing logic was tested to confirm accurate shortest-path computation across the node-edge graph. Additionally, SQLAlchemy ORM operations were examined to ensure consistency in database transactions and to prevent race conditions during concurrent updates. The testing confirmed that all internal decision-making paths and logical conditions were functioning correctly and reliably.

Module-Level and Integration Testing

Unit and integration testing were conducted to validate both individual components and their interaction within the system.

- Unit Testing was performed on modules such as the YOLO-based detection engine, JWT authentication system, and database persistence layer.
- Integration Testing verified the synchronization between the perception layer and the database, ensuring that vehicle detection results in immediate relational updates (SQL UPSERT operations).

The system demonstrated seamless communication between the asynchronous backend and AI inference modules, confirming correct data flow across all components.

TESTING METHODS (Cont'd)

System Logs

The following logs demonstrate real-time interaction between system components:

```
[SYSTEM] Starting SMART LOT: RELATIONAL 2.0 INFRASTRUCTURE...
[SECURE] Verification Phase: Encrypted Hash Match... SUCCESS
[JWT]    Token Generated: Valid Admin Session (Status 200 OK)

[AI]     Frame Processed (112ms) -> Detections Found: 4
[ROI]    Intersection Analysis (Spot_ID: A-012) -> IoA: 0.88 (THRESHOLD EXCEEDED)
[DB]     State Sync: Changing Spot A-012 -> OCCUPIED
[SQL]    UPDATE parking_spots SET status='occupied' WHERE id=12; (COMMIT SUCCESS)

[MATH]   Executing Dijkstra: Entrance_A -> Vacant_Spots
[PATH]   Optimal Found: Node_1 -> Node_5 -> Spot_A-012 (Total Weight: 14.5m)
[HUD]    Streaming Path Coordinates to Driver HUD... (Status 200 OK)
```

These logs confirm correct execution of detection, database synchronization, and routing operations.

1. The "Stitching" Proof (Global Matrix Hub)

This proves your system handles multiple cameras and aligns them into a single global map.

--- TEST CASE: GLOBAL SITE STITCHING ---

[ADMIN] Initializing Matrix Canvas Transformation...

TESTING METHODS (Cont'd)

[STITCH] Zone_B Detected -> Applying Offset Matrix: [X:+1020, Y:-450]
[MATH] Local_ROI (12, 45) -> Transformed Global_ROI (1032, -405)
[SQL] Saving Stitched Perspective to 'global_master_map'... SUCCESS
[RESULT] Global Map Unified: Site 'Main_Lot_V2' -> Active Zones: 3

2. The "Security" Proof (JWT Authorization Guard)

This proves your system is hardened against unauthorized access.
--- TEST CASE: SECURITY GATEKEEPER ---
[PUT] /api/spots/A-005 -> User: 'unauthorized_guest'
[GUARD] Checking Authorization Header... MISSING
[SECURE] JWT Validation Result: FAILURE (Reason: No Token Provided)
[RESULT] 401 UNAUTHORIZED -> Request Terminated by Middleware

3. The "Find My Car" Proof (Reverse-Routing)

This proves your system provides "End-to-End" value for the driver.
--- TEST CASE: VEHICLE RETRIEVAL HUD ---
[GET] /api/find_my_car -> User_ID: 'DRIVER_9982'
[DB] Searching Persistent State... Found: Spot_B-008
[LOGIC] Calculating Reverse Path: Current_Entry -> Spot_B-008
[DIJKSTRA] Node_H -> Node_F -> Node_B -> Spot_B-008 (Weight: 22.4m)
[HUD] Reverse HUD Active: Shortest Path Streamed to Mobile Dashboard

PUBLISHED PAPER

Paper Publication link:
https://ijirt.org/publishedpaper/IJIRT195888_PAPER.pdf

Keywords: Smart Parking System Computer Vision YOLOv8 Parking Occupancy Detection Digital Twin Intelligent Transportation Systems

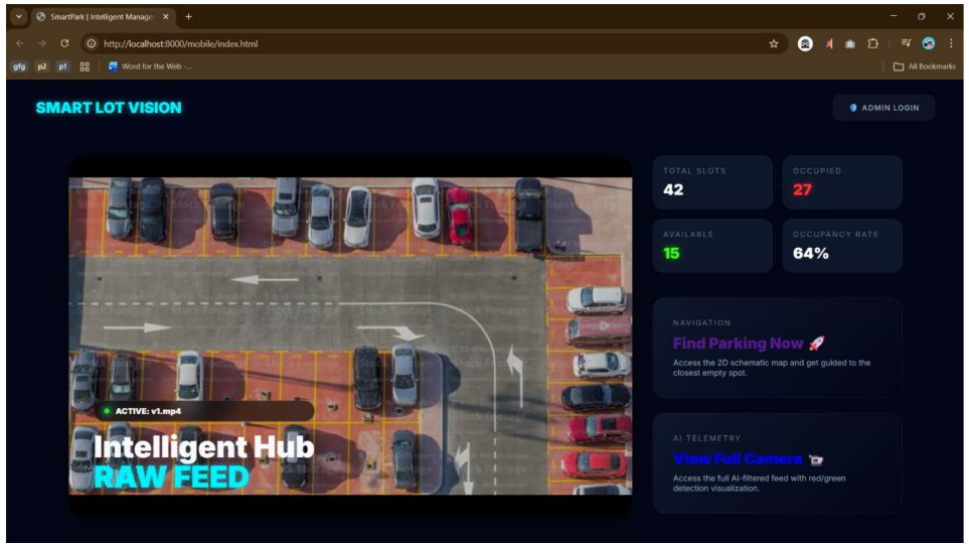
DOI: doi.org/10.64643/IJIRTV12I11-195888-459

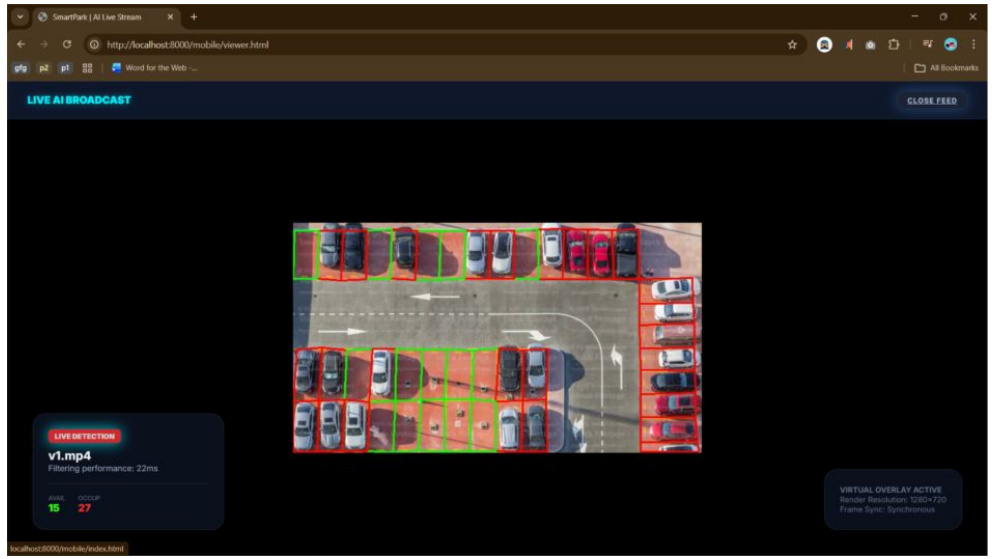
Cite This Article:

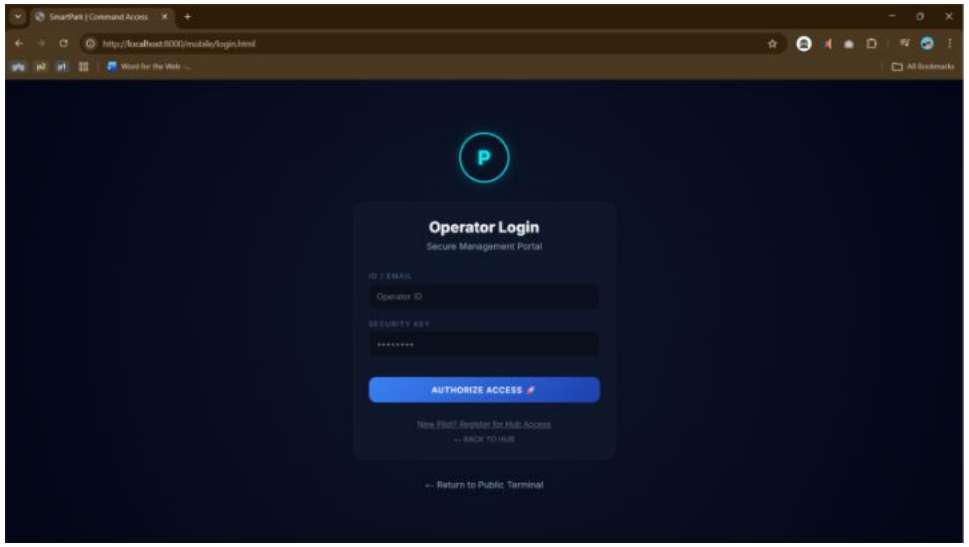
YAMINI, R., & VANDHANA, K., & KUMARI, J. S., & CHARAN, T., & VARDHAN, Y. S., & SRINIVAS, S., & KARTHIKEYA, M. (2026). Smart Lot: Adaptive Parking Management System. International Journal of Innovative Research in Technology (IJIRT).

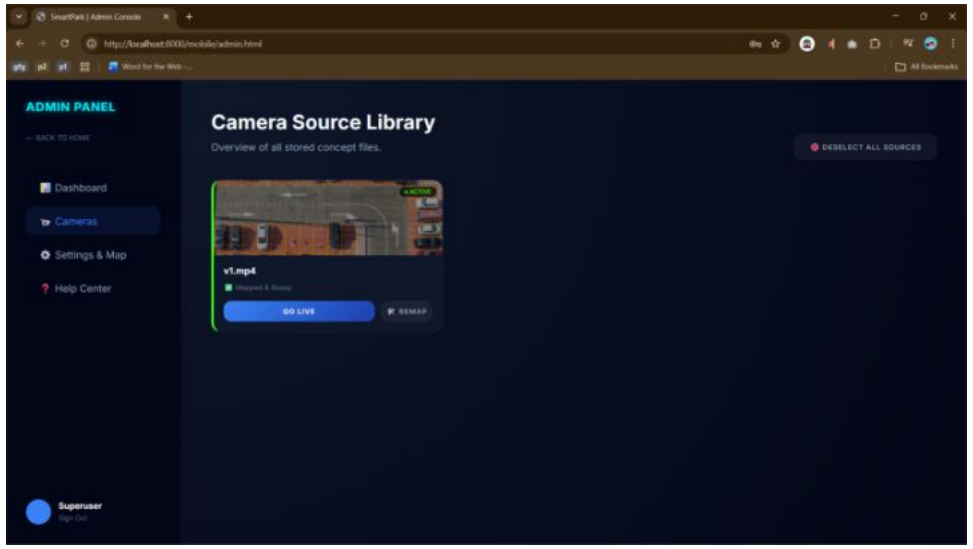
https://doi.org/doi.org/10.64643/IJIRTV12I11-195888-459

OUTPUT SCREENINIG

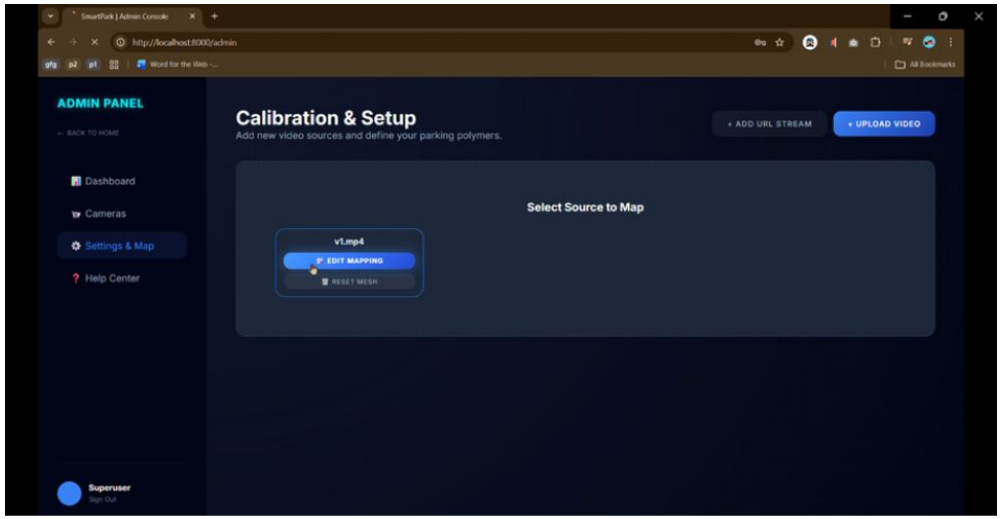


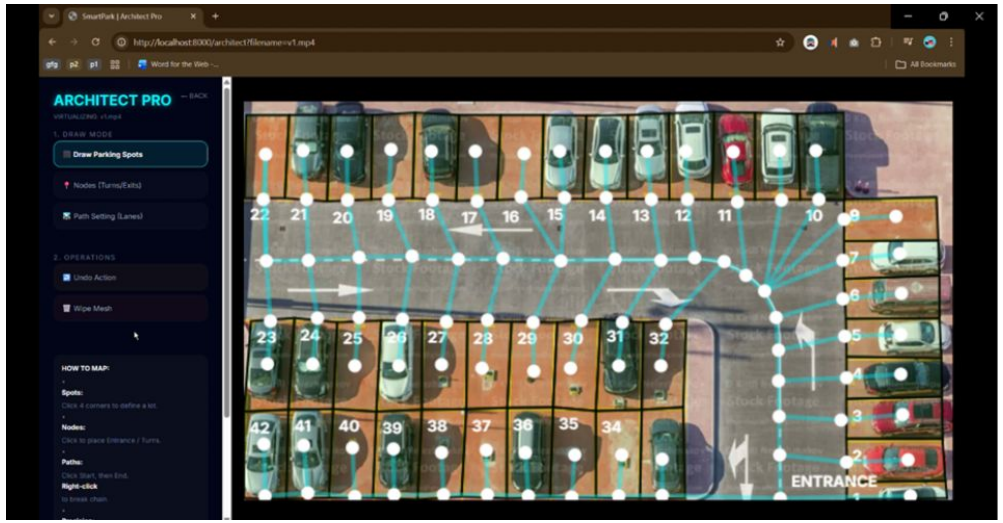


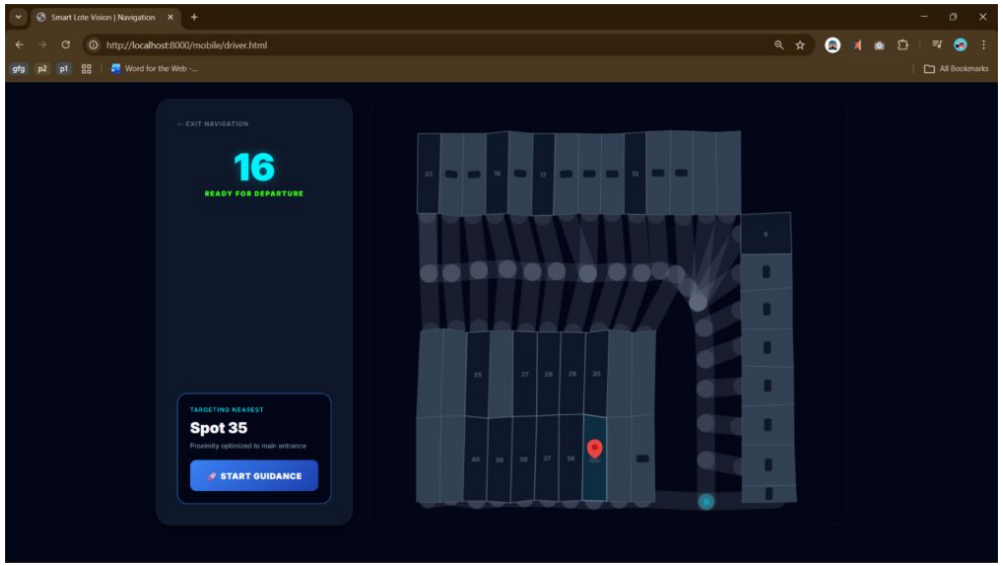


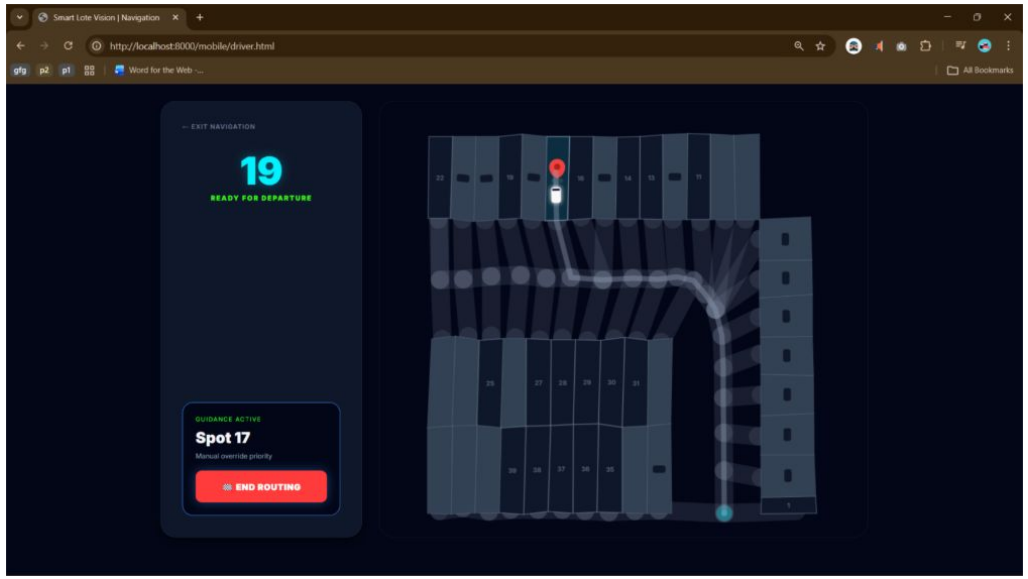


OUTPUT SCREENINIG

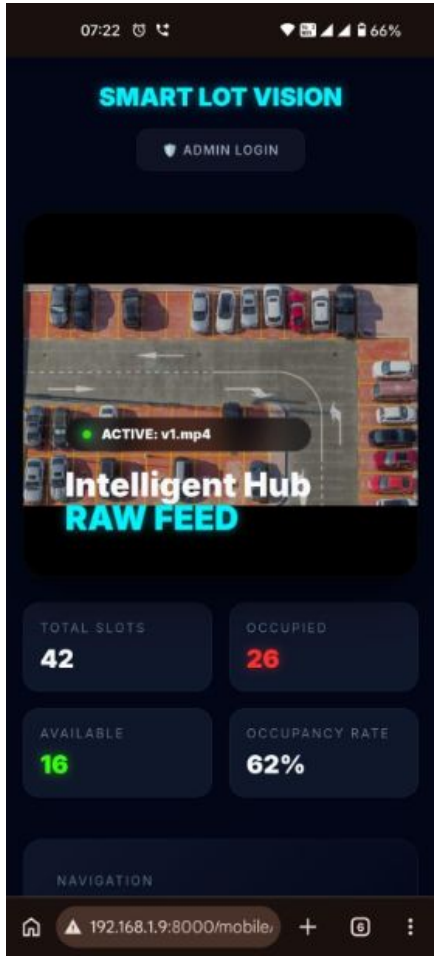


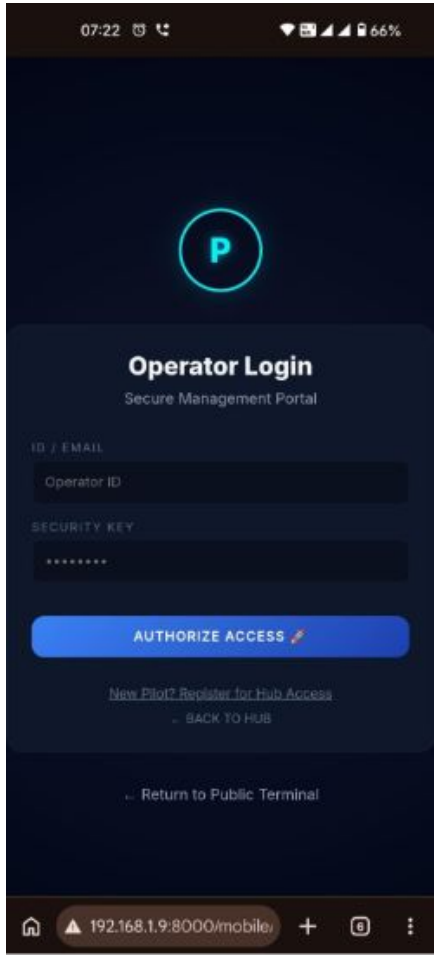


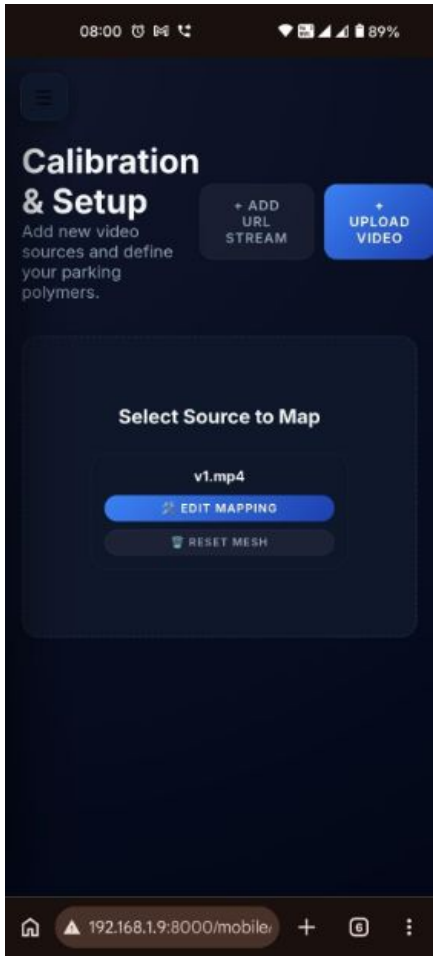


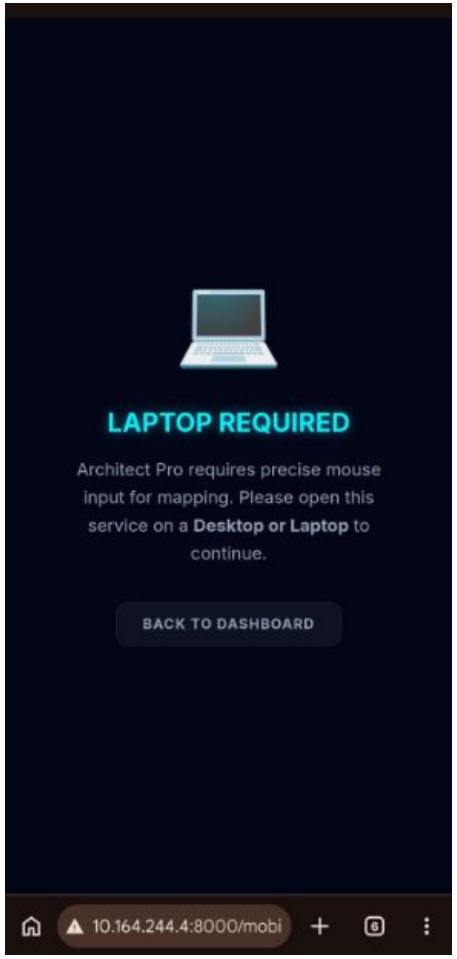


OUTPUT SCREENINIG



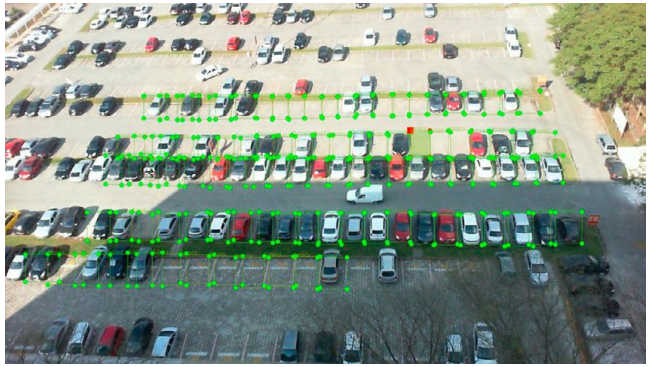






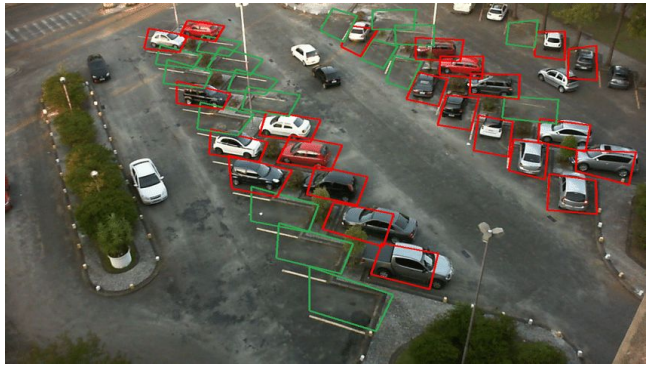
Limitations

The SMART LOT system offers an efficient parking solution, but its current scope is limited by environmental factors, camera setup, network dependency, and initial configuration requirements.



Occlusion and Camera Perspective:
System accuracy depends on camera angle and placement. Low-angle views can cause occlusion, where large vehicles block adjacent spaces, leading to incorrect occupancy detection. A high-angle or near top-down perspective improves accuracy.

Initial Configuration Requirements:
The system requires manual calibration during setup. Administrators must define Regions of Interest (ROIs) for each parking slot and align multiple camera feeds in multi-zone environments to ensure accurate spatial mapping.



Dynamic Shadow Interference :

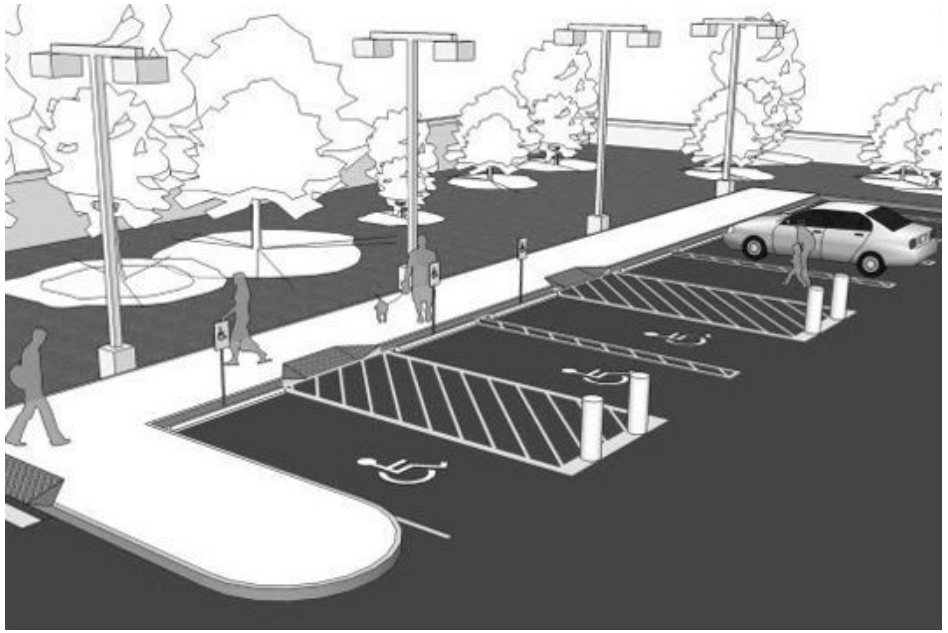
The Limitation: In very large outdoor lots, moving shadows from trees or nearby skyscrapers can occasionally "confuse" the neural detector if the shadow looks remarkably like a vehicle's silhouette.

Scope and Future Considerations:
These limitations indicate opportunities for improvement, including automated calibration, better low-light performance, and more robust system design for consistent operation across varied environments.

Future Work

The SMART LOT system provides a solid base for intelligent parking management, but further enhancements can enable large-scale real-world deployment. Future improvements focus on accessibility, automation, security, and integration with advanced mobility systems.

- **Omni-Viewport Expansion:** Extend support beyond desktop and mobile to a unified multi-device architecture, including optimized tablet interfaces and integration with in-vehicle infotainment systems for real-time, hands-free parking guidance.
- **Intelligent Administrative Systems:** Incorporate automated monitoring, real-time telemetry, and intelligent rule-based mechanisms to manage parking dynamically. Advanced analytics can provide insights into usage patterns and improve decision-making.
- **Security and Data Protection:** Enhance system security through advanced encryption, secure communication channels, and end-to-end data protection. Implement privacy-preserving techniques such as data anonymization.
- **Global Geolocation Integration:** Integrate GPS-based mapping to enable interaction with external navigation platforms, allowing users to locate and navigate to parking facilities on a city-wide scale.
- **Predictive Intelligence:**Use historical data to forecast parking availability and provide proactive recommendations, improving efficiency in high-demand scenarios.
- **Autonomous System Integration:** Enable compatibility with autonomous vehicles by supporting automated parking guidance, positioning SMART LOT within future intelligent transportation systems.



Conclusion

The **SMART LOT** is a vision-based parking management system that leverages existing CCTV/IP cameras to monitor and manage parking spaces, eliminating the need for costly hardware sensors. By integrating advanced object detection with geometric and spatial analysis, the system can accurately determine parking occupancy even under challenging lighting, weather, and perspective conditions.

It uses multi-zone coordination and graph-based routing for efficient navigation in complex parking areas. The system operates in real time with low latency and efficient resource use. Results show high accuracy, reliable performance, and reduced costs, with a modular and scalable design adaptable to various parking setups.

Beyond current applications, SMART LOT provides a strong foundation for integration with smart city ecosystems, supporting future enhancements such as predictive parking analytics, traffic flow optimization, and seamless integration with intelligent transportation systems.

THANK YOU